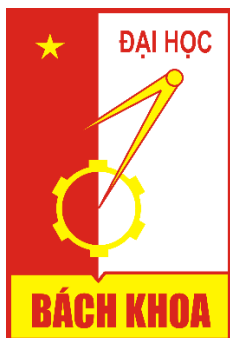


ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

----- ∞  ∞ -----



BÁO CÁO KỸ THUẬT
Đề tài: Phân loại hình ảnh động vật

Giảng viên: **Nguyễn Kiêm Hiếu**

Sinh viên: Nguyễn Thành Long

MSSV: 20235142

Hà Nội, năm 2025

MỞ ĐẦU.....	4
CHƯƠNG I. MÃ NGUỒN, TẬP DỮ LIỆU VÀ CÁC ĐỘ ĐO ĐÁNH GIÁ.....	5
1. Tập dữ liệu	5
2. Minh chứng về kết quả	5
3. Các độ đo đánh giá :.....	7
a. Accuracy [1]	7
b. Precision [1]	7
c. F1-Score [1]	7
d. Recall (Sensitivity) [1].....	7
e. Confusion Matrix [1].....	7
f. Các dữ liệu khác [2]	8
CHƯƠNG II. CẤU TRÚC MÃ NGUỒN, HYPERPARAMETERS, LOSS FUNCTION, OBJECTIVE FUNCTION	9
1. Mã nguồn	9
2. Cấu trúc mã nguồn.....	9
3. Hyperparameters (siêu tham số)	11
4. Hàm mất mát, Hàm mục tiêu.....	12
a. Hàm mất mát Cross-Entropy Loss for multiclass [3]	12
b. Hàm mục tiêu Regularized Minimization (AdamW) [4] [5] [6]	12
3. Lưu ý với các model	12
CHƯƠNG III. CÁC PHƯƠNG PHÁP THỬ NGHIỆM CHO BÀI TOÁN.....	13
I. Các kiến trúc riêng biệt.....	13
1. Custom CNN (Convolutional Neural Network) [8] [9] [10]	13
2. Custom Transformer [14] [15] [16].....	14
II. Các kiến trúc pre-trained	15
1. ResNet50 [14] [25] [26]	15
2. EfficientNetV2	16
3. MobileNetV3 [33]	16

4. DeiT (Data-efficient Image Transformer) [34] [35].....	16
5. VGG16 [36] [37] [38]	16
CHƯƠNG IV. HIỆU NĂNG, ĐƯỜNG HỌC VÀ SUY LUẬN.....	18
1. Custom CNN.....	18
2. Custom Transformer	18
3. ResNet50.....	19
4. EfficientNetV2	20
5. MobileNetV3.....	21
6. DeiT	21
7. VGG16.....	22
CHƯƠNG V. THIÊN HƯỚNG PHÁT TRIỂN.....	23
KẾT LUẬN	24
TÀI LIỆU THAM KHẢO.....	25

MỞ ĐẦU

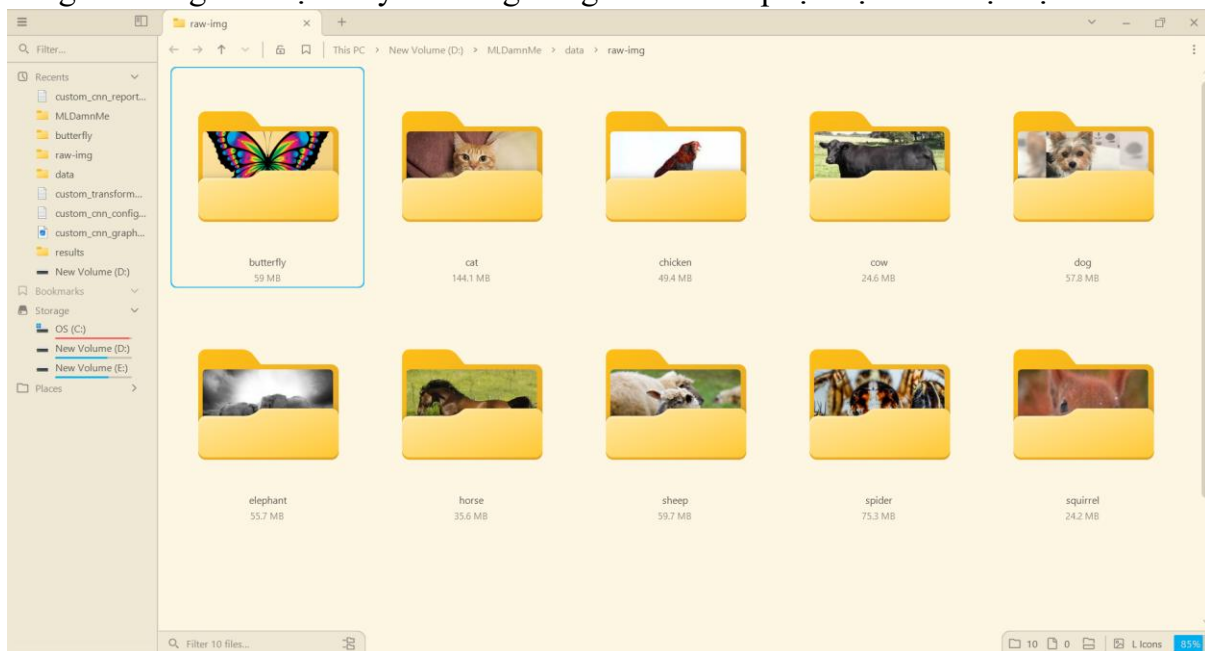
Bài tập lớn này nghiên cứu so sánh hiệu suất của các kiến trúc học sâu khác nhau đối với phân loại hình ảnh đa lớp sử dụng bộ dữ liệu Animals-10, bao gồm khoảng 26.000 hình ảnh thuộc 10 loài động vật riêng biệt. Thí nghiệm bao quát một phạm vi rộng các phương pháp khác nhau, từ các Mạng Nơ-ron Tích chập (CNN) và Vision Transformers (ViT) được tự thiết kế, đến các mô hình tiêu chuẩn được huấn luyện trước như ResNet50, EfficientNetV2 và MobileNetV3. Một trọng tâm chính của việc triển khai này là tối ưu hóa phần cứng, tận dụng các kỹ thuật như Độ chính xác Hỗn hợp Tự động (AMP) và Gradient Checkpointing để đảm bảo quá trình huấn luyện diễn ra hiệu quả trên thiết bị có cấu hình thấp (cụ thể là Laptop NVIDIA RTX 3060 với 6GB VRAM). Ngoài việc huấn luyện và đánh giá mô hình, dự án thiết lập một quy trình đầu cuối (end-to-end) hoàn chỉnh, bao gồm giao diện người dùng đồ họa dựa trên Python cho phép tương tác thời gian thực, với mục đích làm nổi bật sự đánh đổi giữa độ phức tạp kiến trúc, mức sử dụng tài nguyên tính toán và độ chính xác phân loại trong các tác vụ thị giác máy tính thực tế.

CHƯƠNG I. MÃ NGUỒN, TẬP DỮ LIỆU VÀ CÁC ĐỘ ĐO ĐÁNH GIÁ

1. Tập dữ liệu

Tập dữ liệu là Animals-10 được lấy từ đường dẫn sau :
<https://www.kaggle.com/datasets/alessiocrrado99/animals10>

Tập dữ liệu có hơn 28 nghìn ảnh được chia ra thành 10 classes khác nhau gồm các loài động vật như mèo, gà, bò, cừu... Tập dữ liệu ban đầu có tên các classes bằng tiếng Ý nhưng đã được thay đổi sang tiếng Anh nhằm phục vụ cho thuận lợi.

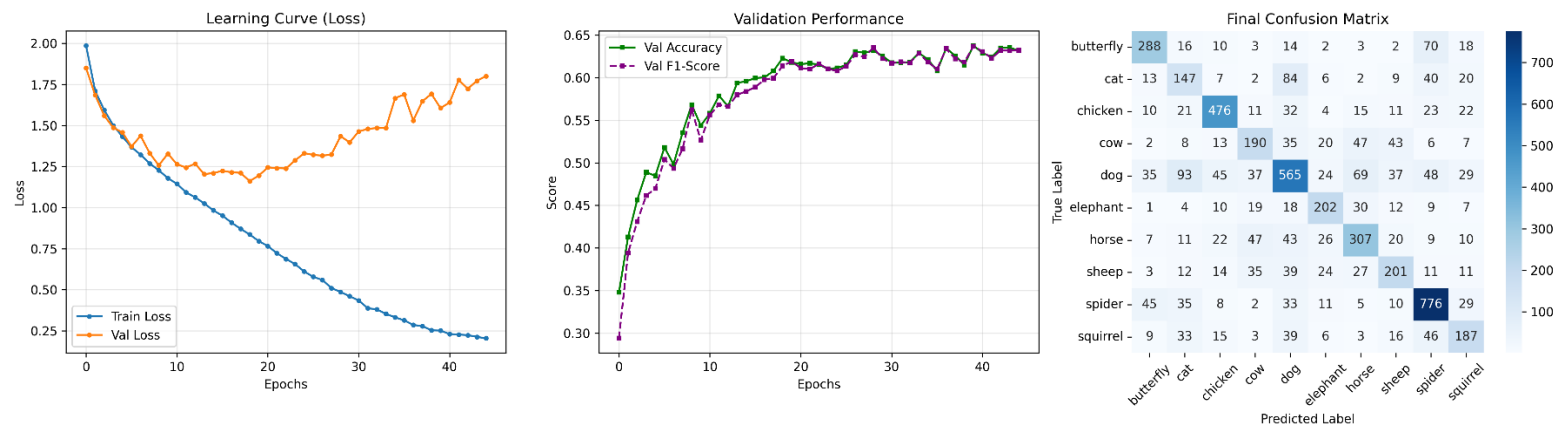


Tỉ lệ ảnh dùng cho training/validation là 80/20. (Nguồn file dataset.py)

```
train_size = int(0.8 * len(full_dataset))
val_size = len(full_dataset) - train_size
train_ds, val_ds = random_split(full_dataset, [train_size, val_size])
```

2. Minh chứng về kết quả

Kết quả của training được lưu trong folder results, với file ảnh định dạng PNG có hiện đường học (Learning Curve), hiệu suất kiểm thử (Validation Performance) và ma trận nhầm lẫn (Confusion Matrix) giữa các classes khác nhau.



Ảnh 1. Ví dụ cho CustomTransformer

File định dạng txt lưu các thông số sau training như precision, recall, F1-score, support, accuracy và macro/weighted avg.

	precision	recall	f1-score	support
butterfly	0.6973	0.6761	0.6865	426
cat	0.3868	0.4455	0.4141	330
chicken	0.7677	0.7616	0.7647	625
cow	0.5444	0.5121	0.5278	371
dog	0.6264	0.5754	0.5998	982
elephant	0.6215	0.6474	0.6342	312
horse	0.6043	0.6116	0.6079	502
sheep	0.5568	0.5332	0.5447	377
spider	0.7476	0.8134	0.7791	954
squirrel	0.5500	0.5238	0.5366	357
accuracy			0.6377	5236
macro avg	0.6103	0.6100	0.6095	5236
weighted avg	0.6376	0.6377	0.6369	5236

Ảnh 2. Ví dụ cho CustomTransformer.

File JSON còn lại miêu tả các hyperparameters cho việc training.

3. Các độ đo đánh giá :

a. Accuracy [1]

Được tính bằng tỉ lệ giữa các dự đoán đúng (TP, TN) chia cho toàn bộ các dự đoán.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Accuracy có thể cho ta hiểu nhanh, nhưng nó có thể gây hiểu nhầm trong trường hợp tập dữ liệu mất cân bằng. Ví dụ, trong một tập dữ liệu có 90% thuộc lớp A và 10% thuộc lớp B, một mô hình chỉ dự đoán lớp A vẫn đạt được độ chính xác 90%, nhưng sẽ không phát hiện được bất kỳ mẫu nào thuộc lớp B.

b. Precision [1]

Được tính bằng tỉ lệ giữa các dự đoán positive đúng chia cho toàn bộ các dự đoán đúng.

$$\text{Precision} = \frac{TP}{TP + FP}$$

Precision hữu ích khi dự đoán positive sai có hậu quả lớn, chẳng hạn như trong chuẩn đoán hình ảnh y khoa hay hệ thống xe tự lái, nơi việc dự đoán đúng trong khi thực tế không có thể dẫn đến những hậu quả khôn lường. **Precision** giúp đảm bảo rằng khi mô hình dự đoán một kết quả positive, thì khả năng dự đoán đó là đúng là cao.

c. F1-Score [1]

Được tính bằng harmonic mean của Precision và Recall. Chỉ số này hữu ích khi chúng ta cần sự cân bằng giữa precision và recall, vì nó kết hợp cả hai vào một giá trị duy nhất. F1-score cao cho thấy mô hình hoạt động tốt trên cả hai độ đo này.

$$\text{F1-score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

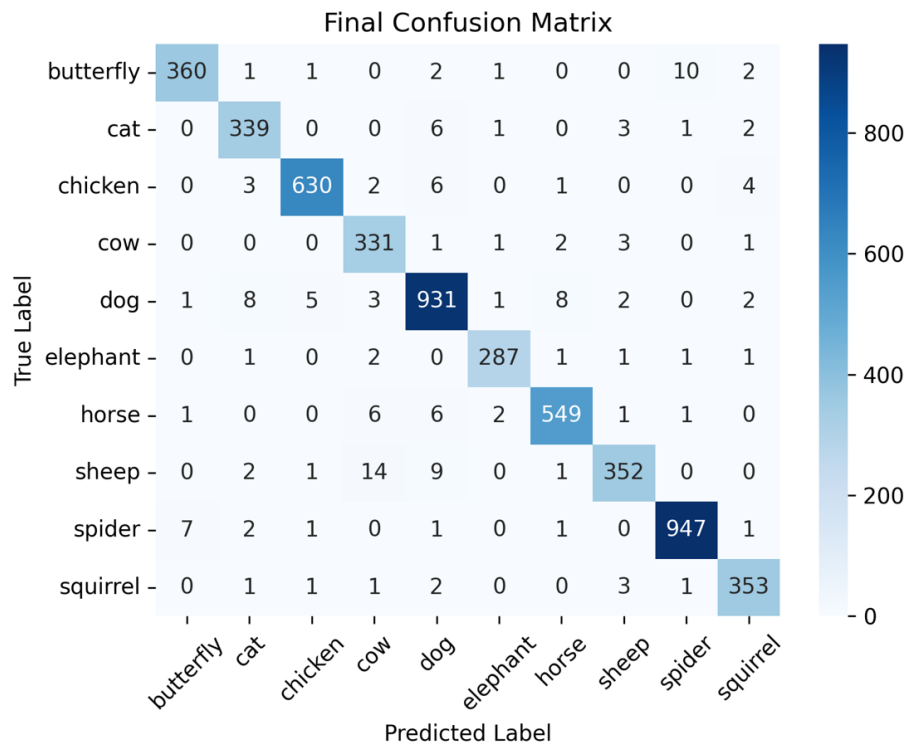
d. Recall (Sensitivity) [1]

Được tính bằng tỉ lệ giữa các dự đoán positive đúng chia cho toàn bộ các dự đoán của một class được xét. Quan trọng trong các trường hợp mà yêu cầu mọi dự đoán đúng như chuẩn đoán hình ảnh y khoa.

$$\text{Recall} = \frac{TP}{TP + FN}$$

e. Confusion Matrix [1]

Trong dự án này, Confusion Matrix là ma trận thể hiện biểu đồ trực quan so sánh nhãn thực (ground truth) với nhãn dự đoán (prediction), cho phép xác định các mẫu sai phân loại (misclassification) cụ thể.



Ảnh 3. Ví dụ về Confusion Matrix của ResNet50

f. Các dữ liệu khác [2]

- **Support** : Số lượng ảnh của một class ở trong Validation test. Có thể sử dụng để đưa ra kết luận về sự cân bằng của test dataset. Nếu Cat có support = 500 và Spider có support = 50, thì tập dữ liệu sẽ bị mất cân bằng. Độ chính xác cao trên lớp Cat sẽ ảnh hưởng nhiều hơn đến điểm accuracy tổng thể, nhưng mô hình có thể hoàn toàn thất bại trong việc dự đoán lớp Spider nếu chỉ nhìn vào accuracy tổng.
- **Macro Average** : Giá trị trung bình của các điểm số trên tất cả các lớp, trong đó mọi lớp đều được đối xử ngang nhau, bất kể mỗi lớp có bao nhiêu ảnh.

$$\text{macroAvg} = \frac{\text{Score}_a + \text{Score}_b + \dots + \text{Score}_x}{\text{numberOfClasses}}$$

- **Weighted Average** : Cũng là giá trị trung bình nhưng được điều chỉnh theo số lượng ảnh trong mỗi class.

$$\text{weightedAvg} = \frac{\text{Score}_a \times n_a + \text{Score}_b \times n_b + \dots + \text{Score}_x \times n_x}{\sum \text{instancesOfEachClasses}}$$

CHƯƠNG II. CẤU TRÚC MÃ NGUỒN, HYPERPARAMETERS, LOSS FUNCTION, OBJECTIVE FUNCTION

1. Mã nguồn

Mã nguồn của bài tập lớn được lưu trữ ở đường dẫn Github sau :

<https://github.com/longnguyendevone/MLEAnimalClass>

Lưu ý : Dự án này yêu cầu môi trường Python riêng qua Anaconda và khuyến khích được thực hành trên phần cứng có GPU hỗ trợ CUDA.

Các bước sau cần được thực hiện với Anaconda Prompt :

```
conda create --name animals10_env python=3.9
conda activate animals10_env
conda install pytorch torchvision torchaudio pytorch-cuda=12.1 -c pytorch -c nvidia
pip install matplotlib seaborn scikit-learn pandas tqdm gradio timm
pip install gradio==3.50.2
code --
```

Sau dòng lệnh cuối Visual Studio Code sẽ được mở với môi trường Python riêng đã được cài đặt.

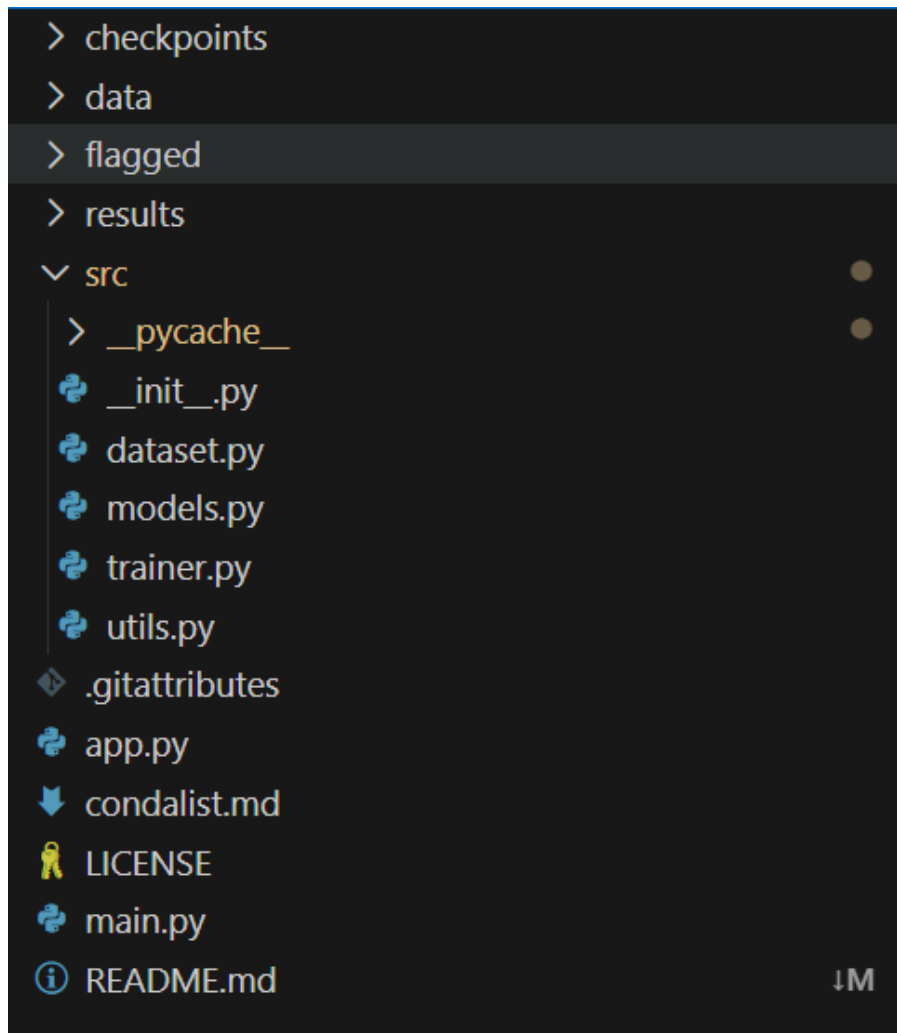
Các command sau được sử dụng để train các model :

```
Custom CNN: python main.py --models custom_cnn --epochs 15 --batch_size 32 --lr 0.001
Custom Transformer: python main.py --models custom_transformer --epochs 45 --batch_size 64 --lr 0.0005
MobileNetV3: python main.py --models mobilenet --epochs 30 --batch_size 64 --no_checkpointing
EfficientNetV2: python main.py --models efficientnetv2 --epochs 30 --batch_size 64
ResNet50: python main.py --models resnet50 --epochs 30 --batch_size 64
VGG16: python main.py --models vgg16 --epochs 30 --batch_size 32 --freeze
deit: python main.py --models deit --epochs 30 --batch_size 32 --lr 0.0005
```

Sử dụng command sau để thực thi giao diện của dự án :

```
python app.py (Và truy cập địa chỉ được đưa ra sau đó)
```

2. Cấu trúc mã nguồn



Các folder và file chính của mã nguồn cũng như các thư viện được sử dụng.

checkpoints	Lưu các model sau khi được pre-trained trên bộ dữ liệu.
data	Bộ dữ liệu bao gồm 10 folder khác nhau.
results	Lưu trữ các file PNG, TXT và JSON về kết quả và các hyperparameters.
dataset.py	Load , phân chia (train/val) và pre-process dữ liệu ảnh.
models.py	Định nghĩa kiến trúc các mô hình (Custom & Pre-trained).
trainer.py	Thực thi huấn luyện, tính loss và cập nhật trọng số.
utils.py	Vẽ biểu đồ, tính toán chỉ số đánh giá và xuất báo cáo.
app.py	Chạy giao diện web (GUI) để dự đoán hình ảnh thực tế.
main.py	Điều phối luồng chạy chính và quản lý tham số cấu hình.
torch	Thư viện của PyTorch dùng để định nghĩa tensor và neural networks.

torchvision	Được sử dụng cho các tập dữ liệu, biến đổi hình ảnh (tăng cường dữ liệu) và kiến trúc mô hình.
timm	Tải các mô hình như EfficientNetV2, MobileNetV3... đã được huấn luyện trước.
scikit-learn	Được sử dụng trong utils.py để tính toán Precision, Recall, F1-Score, Accuracy và Confusion Matrix.
numpy	Thư viện tính toán.
matplotlib	Được sử dụng để tạo và lưu các đồ thị và hình ảnh ma trận nhầm lẫn.
gradio	Thư viện để xây dựng GUI web (apps.py)
NVIDIA CUDA	Cho phép training và inferencing trên GPU.

3. Hyperparameters (siêu tham số)

Model	Epochs	Batch Size	Learning Rate	Lưu ý
Custom CNN	15	32	0.001	Thiết lập cơ bản.
Custom Transformer	45	64	0.0005	LR chia nửa để tránh gradient explosions; Epochs số lượng lớn để có thể học thêm.
MobileNetV3	30	64	0.001	--no_checkpointing do phần cứng có thể chạy với tốc độ cao.
EfficientNetV2	30	64	0.001	
ResNet50	30	64	0.001	
VGG16	30	32	0.001	--freeze để tránh tràn bộ nhớ 6GB VRAM.
DeiT Tiny	30	32	0.0005	LR chia nửa để cân bằng attention.

4. Hàm mất mát, Hàm mục tiêu

a. Hàm mất mát Cross-Entropy Loss for multiclass [3]

```
nn.CrossEntropyLoss()
```

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C (y_{ij} \times \log p_{ij})$$

N là cỡ batch (32, 64,...).

C là số classes (10).

y_{ij} là giá trị binary thể hiện c là phân loại đúng với dự đoán i .

p_{ij} là xác suất dự đoán (sau Softmax) rằng quan sát i thuộc về lớp c .

Cross-Entropy phạt các dự đoán dựa trên mức độ tự tin. Nếu mô hình dự đoán “Dog” với độ tin cậy 90% và kết quả đúng là Dog, thì loss gần bằng 0. Ngược lại, nếu mô hình chỉ dự đoán “Dog” với độ tin cậy 10%, thì loss sẽ rất lớn. Điều này buộc mô hình không chỉ dự đoán “đúng”, mà còn phải đúng với độ tự tin cao, từ đó tạo ra các ranh giới quyết định mạnh mẽ giữa 10 lớp động vật.

b. Hàm mục tiêu Regularized Minimization (AdamW) [4] [5] [6]

```
optimizer = optim.AdamW(model.parameters(), lr=args.lr)
```

θ là model weights.

$$J(\theta) = L_{CE}(\theta) + \frac{\lambda}{2} \|\theta\|^2$$

L_{CE} là Cross-Entropy loss ở trên.

λ là Hệ số weight decay (giá trị mặc định trong AdamW thường là 0.01).

SGD chuẩn (Stochastic Gradient Descent) sử dụng một learning rate duy nhất cho tất cả các tham số. Trong khi đó, Adam tính toán learning rate thích nghi riêng cho từng tham số. Điều này rất quan trọng đối với Transformer (như DeiT, Custom Transformer), vốn có các lớp attention phức tạp và dễ gặp vấn đề tiêu biến hoặc bùng nổ gradient. Adam chuẩn triển khai weight decay chưa đúng (trộn lẫn nó với gradient). AdamW tách biệt (decouple) weight decay, áp dụng trực tiếp lên trọng số. Cách này giúp tổng quát hóa tốt hơn (giảm overfitting), điều này thể hiện rõ qua việc ResNet và EfficientNet của duy trì accuracy cao trên tập validation.

3. Lưu ý với các model

- VGG16 $\min_{\theta_{head}} L_{CE}(\theta_{backbone}, \theta_{head})$

Bộ tối ưu chỉ cập nhật trọng số của Classifier Head (θ_{head}). Trọng số của Backbone ($\theta_{backbone}$) được coi là hằng số. Hàm mục tiêu thực chất đang tìm tổ hợp tuyến tính tốt nhất của các đặc trưng đã học từ ImageNet để phân loại bộ dữ liệu Animals-10. [7]

- Các model khác $\min_{\theta_{all}} L_{CE}(\theta_{all})$

Bộ tối ưu đã cập nhật tất cả các tham số trong mạng, cho phép các bộ trích xuất đặc trưng (backbone) tự điều chỉnh đặc biệt để nhận diện kết cấu và hình dạng của động vật.

CHƯƠNG III. CÁC PHƯƠNG PHÁP THỬ NGHIỆM CHO BÀI TOÁN

Dự án này được thiết kế để đánh giá bảy kiến trúc học sâu khác nhau trên bộ dữ liệu Animals-10 (khoảng 28.000 ảnh, 10 classes) trong môi trường phần cứng hạn chế (GPU NVIDIA RTX 3060 Laptop, 6GB VRAM). Các phương pháp được chia thành hai nhóm: Các kiến trúc riêng biệt (Tự thiết kế và training từ đầu), và các kiến trúc pre-trained (Transfer Learning).

I. Các kiến trúc riêng biệt

1. Custom CNN (Convolutional Neural Network) [8] [9] [10]

CustomCNN là một mạng nơ-ron tích chập (Convolutional Neural Network) nhẹ và hiệu quả, được thiết kế cho bài toán phân loại ảnh. Mô hình này áp dụng các nguyên tắc kiến trúc hiện đại thường thấy trong các mạng di động tiên tiến như MobileNet và EfficientNet, nhằm cân bằng giữa hiệu năng và hiệu quả tính toán.

- **Cấu trúc model: Model được thiết kế theo mô hình stem-body-head.**

1. Stem:

```
self.stem = nn.Sequential(...)
```

Đây là điểm vào của mạng. Lớp này sử dụng tích chập chuẩn 3×3 , theo sau là Batch Normalization và hàm kích hoạt SiLU. Lớp này nhanh chóng mở rộng đầu vào thành bản đồ đặc trưng 32 channels, nhằm trích xuất các đặc trưng mức thấp ban đầu như edges và textures.

2. Body:

```
self.layer1 through self.layer4
```

Phần lõi của mạng bao gồm bốn lớp DSCBlock xếp chồng lên nhau. Mỗi lớp nhân đôi số kênh $32 \rightarrow 64 \rightarrow 128 \rightarrow 256 \rightarrow 512$ đồng thời giảm spatial resolution (stride = 2). Cấu trúc phân cấp này cho phép mạng học được các đặc trưng ngày càng phức tạp và trừu tượng hơn.

3. Head:

```
self.global_pool and self.classifier
```

Thay vì làm phẳng trực tiếp feature map lớn (dẫn đến hàng triệu tham số), mô hình sử dụng **Global Average Pooling** để giảm feature map xuống vector 1×1 (kích thước **512**). Vector này sau đó được đưa vào một lớp Linear duy nhất để thực hiện phân loại cuối cùng.

- **Các phương pháp tối ưu.**

1. Depthwise Separable Convolutions: [11]

Thay vì sử dụng **tích chập chuẩn**, mô hình sử dụng **Depthwise Separable Convolutions**. Cách này chia phép tích chập thành hai bước tính toán đơn giản hơn:

- Depthwise Convolution (self.depthwise): Đây là phép tích chập không gian 3×3 được áp dụng độc lập trên từng kênh đầu vào (groups = in_channels). Lớp này lọc các đặc trưng nhưng không trộn thông tin giữa các kênh.
- Pointwise Convolution (self.pointwise): Đây là phép tích chập 1×1 dùng để trộn (kết hợp) thông tin giữa các kênh.

- Kỹ thuật này giảm mạnh số lượng tham số và số phép toán. Với kernel 3×3 , phương pháp này hiệu quả hơn khoảng 8–9 lần so với tích chập chuẩn, giúp mô hình nhỏ hơn và chạy nhanh hơn đáng kể.

2. Gradient Checkpointing:

```
if self.use_checkpointing ... x = checkpoint.checkpoint(...)
```

Tính năng này cho phép mô hình đánh đổi tốc độ tính toán lấy bộ nhớ (VRAM). Thông thường, các framework học sâu sẽ **lưu trữ toàn bộ các kích hoạt trung gian** trong quá trình *forward pass* để phục vụ cho *backpropagation*. Điều này giảm đáng kể mức sử dụng bộ nhớ đỉnh, cho phép huấn luyện các mô hình lớn hơn hoặc sử dụng batch size lớn hơn trên các GPU có dung lượng RAM hạn chế (6GB).

3. Residual Connections (ResNet-style): [12]

```
out += self.shortcut(x)
```

Khối này cộng trực tiếp đầu vào ban đầu vào đầu ra của các lớp tích chập. Cách làm này giải quyết vấn đề vanishing gradient, cho phép mạng sâu hơn và huấn luyện nhanh hơn mà không làm suy giảm độ chính xác. Cơ chế shortcut tự động xử lý các trường hợp kích thước đầu vào và đầu ra khác nhau (bằng cách sử dụng tích chập 1×1).

4. SiLU (Swish) Activation: [13]

```
nn.SiLU(inplace=True)
```

Thay vì sử dụng hàm kích hoạt ReLU truyền thống, mô hình này sử dụng hàm kích hoạt SiLU (Sigmoid Linear Unit) một hàm trơn, không đơn điệu. Không giống như ReLU có loại bỏ tất cả các giá trị âm (gây ra hiện tượng "dead neuron"), SiLU cho phép một lượng nhỏ thông tin âm đi qua. Điều này thường dẫn đến sự hội tụ tốt hơn và độ chính xác cao hơn.

2. Custom Transformer [14] [15] [16]

CustomTransformer là một phiên bản triển khai gọn nhẹ của Vision Transformer (ViT). Khác với mạng nơ-ron tích chập (CNN) xử lý ảnh thông qua các cửa sổ trượt (bộ lọc), mô hình này xem ảnh như một chuỗi các "patches".

• Cấu trúc model.

1. Tokenizer (Patch Embedding):

```
self.patch_embed using nn.Conv2d
```

Thay vì xử lý từng pixel riêng lẻ, ảnh kích thước 224×224 được chia thành các ô vuông kích thước cố định gọi là patch 16×16 . Ảnh được chuyển thành một chuỗi gồm 196 token (grid 14×14), sau đó các token này được ánh xạ vào không gian vector có kích thước 256 (embed_dim).

2. Transformer Encoder:

```
self.encoder
```

Stack 6 lớp này sử dụng cơ chế Self-Attention, cho phép mỗi patch trong ảnh "tương tác" trực tiếp với mọi patch khác ngay lập tức. So với CNN, một lớp CNN chỉ nhìn thấy các pixel lân cận, trong khi lớp Transformer này nắm bắt được mối quan hệ toàn cục của toàn bộ ảnh (ví dụ: liên hệ cái đuôi ở góc dưới bên trái với cái tai ở góc trên bên phải) ngay từ lớp đầu tiên..

3. Classifier:

```
self.cls_token and self.head
```

Một “Class Token” đặc biệt có thể học được được thêm vào chuỗi. Khi thông tin lan truyền qua các lớp, token này sẽ aggregate thông tin đại diện cho toàn bộ ảnh. Cuối cùng, chỉ token duy nhất này được đưa vào lớp Linear để dự đoán lớp động vật..

- **Các phương pháp tối ưu.**

1. *Pre-Normalization*: [\[17\]](#) [\[18\]](#) [\[19\]](#)

```
TransformerEncoderLayer(..., norm_first=True)
```

Trong bài Transformer gốc “Attention Is All You Need”, chuẩn hóa (normalization) được đặt sau khối attention (Post-Norm). Mô hình này đặt chuẩn hóa trước (Pre-Norm). Pre-Norm tạo ra direct flow cho gradient lan truyền trong mạng (tương tự ResNet). Điều này giúp quá trình huấn luyện ổn định hơn đáng kể, cho phép mô hình hội tụ mà không cần các schedulers thay đổi learning rate “warm-up” phức tạp như thường thấy ở các Transformer dùng Post-Norm.

2. *Convolutional Patch Embedding*: [\[20\]](#) [\[21\]](#)

```
nn.Conv2d(..., kernel_size=16, stride=16)
```

Một naïve implementation sẽ lặp qua từng vị trí trong ảnh để cắt các patch. Đoạn mã này sử dụng tích chập 2D chuẩn với stride bằng đúng kích thước kernel. Đây là một mẹo tối ưu tính toán sử dụng GPU. Cách làm này tận dụng các kernel tích chập CUDA đã được tối ưu, cho phép thực hiện việc chia patch và ánh xạ tuyến tính chỉ trong một phép toán tốc độ cao.

3. *GELU Activation*: [\[13\]](#) [\[22\]](#) [\[23\]](#)

```
activation='gelu'
```

Gaussian Error Linear Units (GeLU) mượt hơn ReLU (vốn có điểm gãy sắc tại 0) và là hàm kích hoạt tiêu chuẩn trong các Transformer hiện đại (BERT, GPT, ViT) vì nó tránh hiện tượng “dead neurons” và giúp cơ chế attention phức tạp mô hình hóa các quan hệ phi tuyến hiệu quả hơn.

4. *Kiến trúc Tiny*: [\[15\]](#)

```
embed_dim=256, num_layers=6, num_heads=4
```

Một ViT-Base tiêu chuẩn có 768 dimensions và 12 layers (khoảng 86 triệu tham số). Mô hình tùy chỉnh này được thu nhỏ xuống còn khoảng 256 dimensions, có thể dễ dàng chạy trên laptop với GPU 6GB. Transformer đòi hỏi rất nhiều dữ liệu. Với tập dữ liệu chỉ khoảng 28.000 ảnh, một ViT lớn sẽ ghi nhớ dữ liệu gần như ngay lập tức. Phiên bản “Tiny” này buộc mô hình phải học các đặc trưng có tính khái quát, thay vì ghi nhớ từng pixel.

II. Các kiến trúc pre-trained

Dự án có năm mô hình pre-trained khác nhau đã được lựa chọn. Các mô hình này không được chọn một cách ngẫu nhiên, mà đại diện cho những cột mốc quan trọng trong lịch sử Học sâu và các triết lý kiến trúc khác nhau. Chúng bao phủ toàn bộ phổ từ các mạng cổ điển, nặng, đến các kiến trúc cho mobile hiện đại, gọn nhẹ, và các Transformer tiên tiến nhất.

1. **ResNet50** [\[14\]](#) [\[25\]](#) [\[26\]](#)

Trước ResNet (2015), các mạng nơ-ron sâu thường gặp vấn đề Vanishing Gradient. Khi tăng số lượng lớp, tín hiệu từ hàm mất mát sẽ dần biến mất trước khi truyền ngược về các lớp đầu trong quá trình *backpropagation*, khiến cho deep networks gần như không thể được huấn luyện. Giải pháp của ResNet là Residual

Block, giới thiệu shortcut/skip connection. Thay vì học trực tiếp ánh xạ $H(x)$, lớp học hàm dư (residual)

$$F(x) = H(x) - x$$

và sau đó cộng lại đầu vào ban đầu:

$$H(x) = F(x) + x.$$

Cách làm này cho phép gradient truyền trực tiếp xuyên suốt mạng, giúp huấn luyện các mạng rất sâu một cách hiệu quả.

2. EfficientNetV2

EfficientNet đã giới thiệu Compound Scaling. Trước đó, hầu hết các nghiên cứu cố gắng cải thiện mô hình bằng cách chỉ tăng độ rộng, hoặc độ sâu, hoặc độ phân giải ảnh. EfficientNet chứng minh rằng tồn tại một mối quan hệ toán học giữa cả ba yếu tố này. [\[27\]](#) [\[28\]](#) [\[29\]](#) [\[30\]](#)

Phiên bản “V2” tập trung tối ưu tốc độ huấn luyện thông qua các lớp Fused-MBConv. Cơ chế này gộp bước mở rộng kênh và tích chập depthwise ở các lớp đầu, nhằm khai thác hiệu quả hơn phần cứng GPU hiện đại. [\[31\]](#) [\[32\]](#)

EfficientNet đại diện cho đỉnh cao của CNN. Mô hình này thường đạt độ chính xác cao nhất trên mỗi tham số được quan sát.

3. MobileNetV3 [\[33\]](#)

MobileNet được thiết kế nhằm tối ưu hiệu năng trên CPU và các thiết bị di động, sử dụng Depthwise Separable Convolutions (cũng được sử dụng trong Custom CNN), trước tiên trích xuất đặc trưng không gian (*Depthwise*), rồi sau đó trộn thông tin giữa các kênh (*Pointwise*). Cách làm này giảm chi phí tính toán khoảng $\approx 8 \times$ so với tích chập chuẩn.

4. DeiT (Data-efficient Image Transformer) [\[34\]](#) [\[35\]](#)

Các ViT tiêu chuẩn đòi hỏi tập dữ liệu rất lớn (hàng triệu ảnh) để huấn luyện. DeiT được lựa chọn vì nó được distillation một cách chuyên biệt để học hiệu quả trên các tập dữ liệu nhỏ hơn (ví dụ như Animals-10 với 28.000 ảnh), cho phép bạn thử nghiệm các cơ chế attention mà không cần đến cấu hình cao.

DeiT không sử dụng tích chập. Thay vì chỉ nhìn vào các pixel lân cận, mô hình chia ảnh thành một chuỗi các patch kích thước 16×16 và sử dụng Self-Attention để mọi patch có thể “tương tác” với mọi patch. Điều này giúp mô hình nắm bắt ngữ cảnh toàn cục (toàn bộ ảnh cùng lúc) tốt hơn so với CNN, nhưng cũng khiến việc huấn luyện trở nên khó khăn hơn.

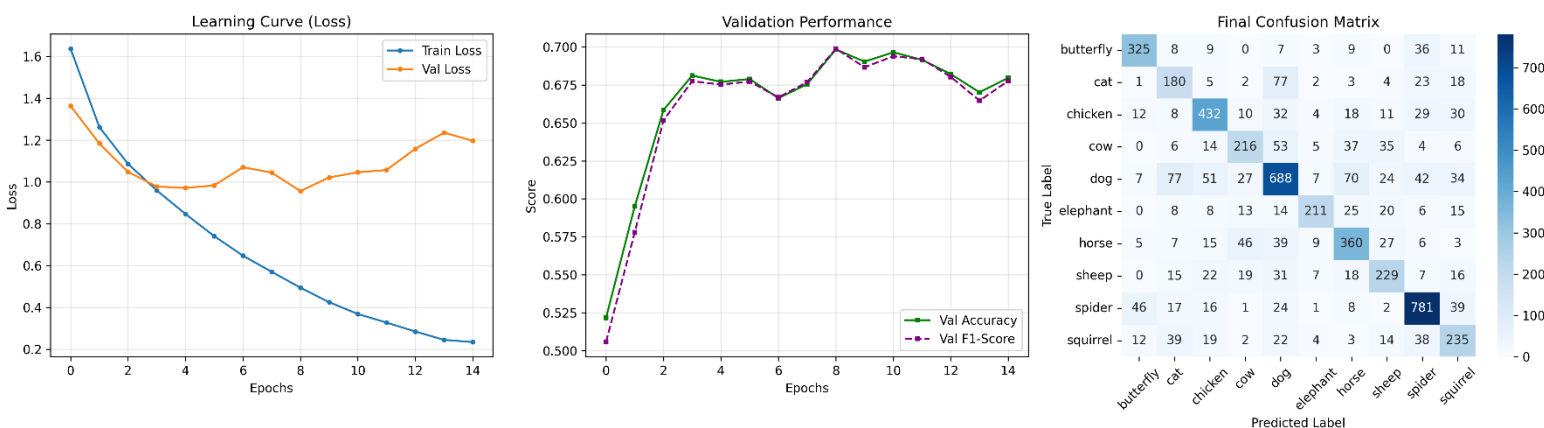
5. VGG16 [\[36\]](#) [\[37\]](#) [\[38\]](#)

VGG là một kiến trúc cũ (2014), nổi tiếng vì kích thước rất lớn và tiêu tốn nhiều bộ nhớ. Mô hình này được đưa vào một cách có chủ đích nhằm kiểm tra giới hạn phần cứng. Nó buộc phải sử dụng kỹ thuật tối ưu bộ nhớ như đóng băng backbone, qua đó chứng minh rằng có thể chạy các mô hình cực lớn trên laptop 6GB VRAM bằng cách điều chỉnh chiến lược huấn luyện.

VGG là một chuỗi “thuần” các lớp tích chập 3×3 . Nó không có các kết nối tắt (skip connection) hay cơ chế attention phức tạp. Phần “Head”: Mô hình có các lớp fully connected rất lớn ở cuối (mỗi lớp rộng 4096 neuron), đây là lý do chính khiến VGG tiêu thụ nhiều VRAM hơn so với các mô hình hiện đại.

CHƯƠNG IV. HIỆU NĂNG, ĐƯỜNG HỌC VÀ SUY LUẬN

1. Custom CNN



Ảnh 4. Custom CNN

- Đường học :

Train loss giảm đều và liên tục theo số epoch → mô hình học rất tốt trên tập huấn luyện và vẫn còn khả năng fit thêm dữ liệu train. Validation loss giảm mạnh ở những epoch đầu (khoảng epoch 0–3), sau đó dao động và có xu hướng tăng dần từ khoảng epoch 4–5 trở đi.

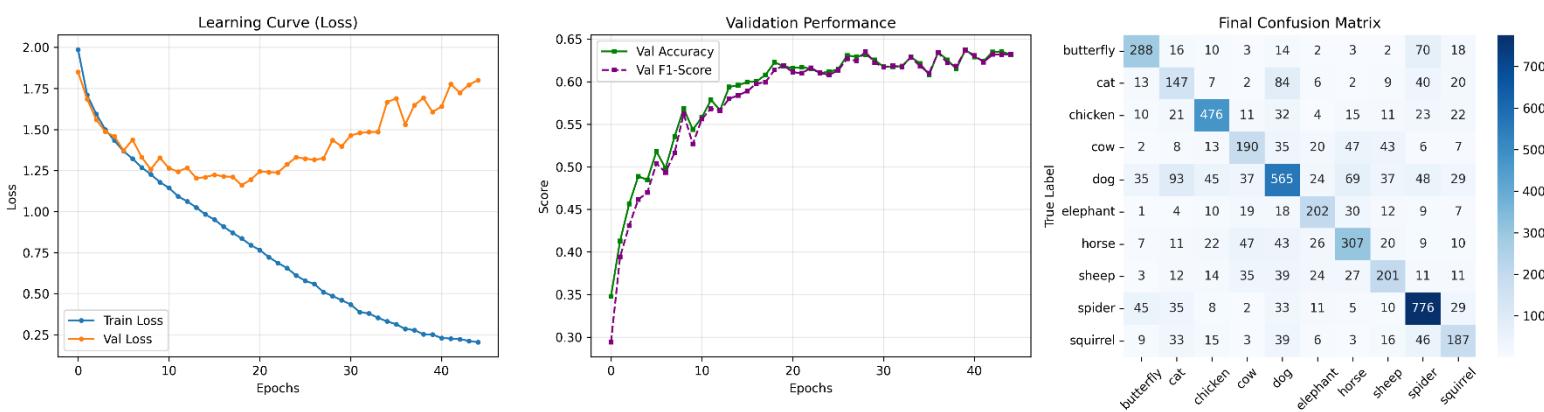
- Hiệu năng trên tập Val :

Mô hình đạt đến một nóc ở mức 70% accuracy. Việc huấn luyện thêm nhiều epoch khó có khả năng cải thiện, vì kiến trúc mô hình quá đơn giản để nắm bắt các đặc trưng phức tạp hơn.

- Ma trận nhầm lẫn :

Đường chéo chính vẫn rõ ràng nhưng nhạt hơn nhiều (ít dự đoán đúng hơn) so với các mô hình tiền huấn luyện. Có sự nhầm lẫn giữa các loài có shape giống nhau (Chó với mèo trong 154 trường hợp).

2. Custom Transformer



Ảnh 5. Custom Transformer

- Đường học :

Không giống các CNN có loss giảm nhanh rồi cong lại, đường loss này gần như là một đường chéo thẳng đi xuống, một đặc điểm cơ bản của Transformer với learning rate chậm.

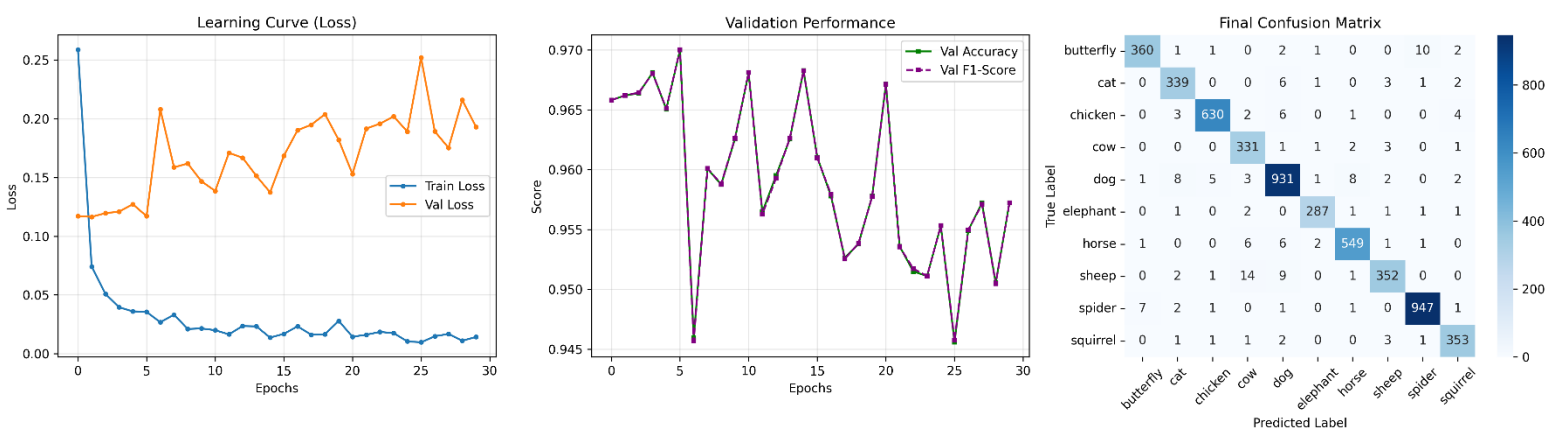
- Hiệu năng trên tập Val :

Accuracy bắt đầu rất thấp (~30%) và tăng đều lên khoảng 63%. Quá trình học ổn định, cho thấy learning rate thấp (0.0005) là lựa chọn đúng để giữ huấn luyện ổn định. Sau 45 epoch, validation accuracy vẫn đang tăng rồi tụt lại ở khoảng 0.6-0.65.

- Ma trận nhầm lẫn :

Có lỗi rải rác khắp nơi (ví dụ 35 ảnh Dog bị phân loại thành Butterfly). Điều này cho thấy mô hình vẫn đang “đoán” trên nhiều ảnh và chưa hình thành đầy đủ các bản đồ attention mạnh mẽ cho hình dạng động vật.

3. ResNet50



Ảnh 6. ResNet50

- Đường học :

Training loss giảm rất nhanh và gần như hoàn hảo, từ khoảng 0.25 xuống gần 0.00 chỉ sau ~5 epoch. Validation loss ổn định rất sớm (khoảng epoch 3–4) và dao động trong khoảng 0.15–0.25. Nó không tiếp tục giảm đáng kể sau vài epoch đầu, cho thấy mô hình đã nhanh chóng đạt tới giới hạn năng lực tối đa đối với tập dữ liệu. Khoảng cách giữa Train và Validation phản ánh overfitting nhẹ, điều này là bình thường đối với các mô hình có năng lực cao.

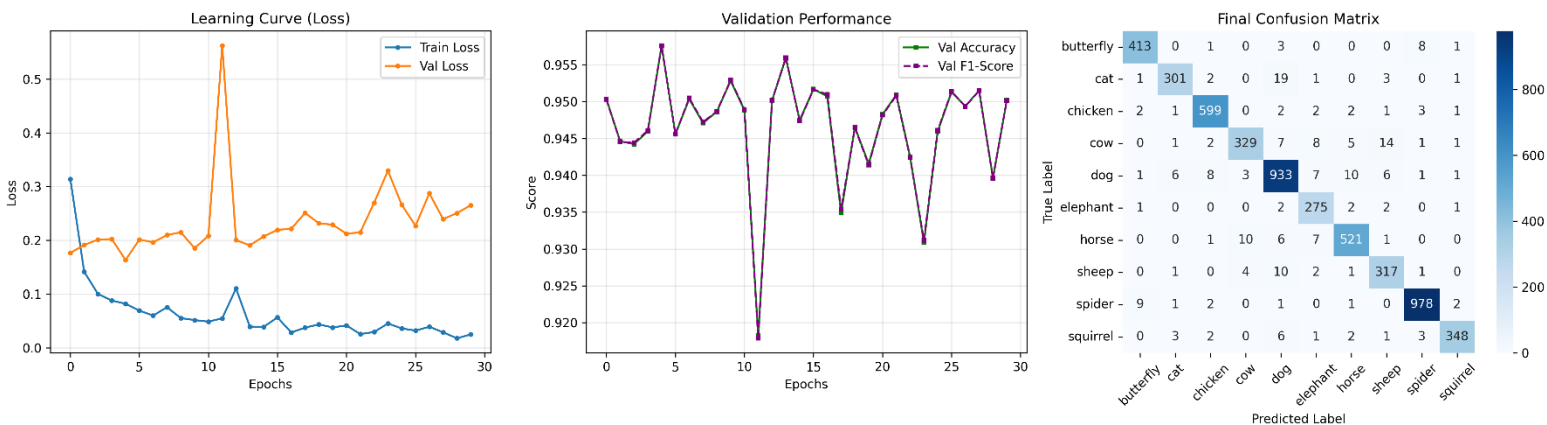
- Hiệu năng trên tập Val :

Validation Accuracy và F1-Score tương quan với nhau và duy trì ở mức rất cao (>95%). Xuất hiện những đỉnh dạng răng cưa trong các chỉ số (ví dụ tụt tại epoch 6 và 25).

- Ma trận nhầm lẫn :

Đường chéo rất nét, gần như xanh đậm hoàn toàn, cho thấy tỷ lệ dự đoán đúng cực cao. Lỗi thì gần như không đáng kể. Sự nhầm lẫn đáng chú ý duy nhất là rất nhỏ (ví dụ 5–8 ảnh Dog bị phân loại sai). Mô hình phân biệt gần như hoàn hảo các lớp có hình dạng tương tự (như Horse và Cow).

4. EfficientNetV2



Ảnh 7. EfficientNetV2

- Đường học :

Đặc điểm nổi bật nhất là đỉnh tăng rất mạnh của Validation Loss (kèm theo sự sụt giảm accuracy) tại epoch 12. Điều này nhiều khả năng do gradient explosion hoặc một “batch xấu”, khiến bộ tối ưu tạm thời đẩy trọng số đi sai hướng. Điều quan trọng là mô hình phục hồi ngay ở epoch tiếp theo, không bị kẹt ở cực tiểu cục bộ.

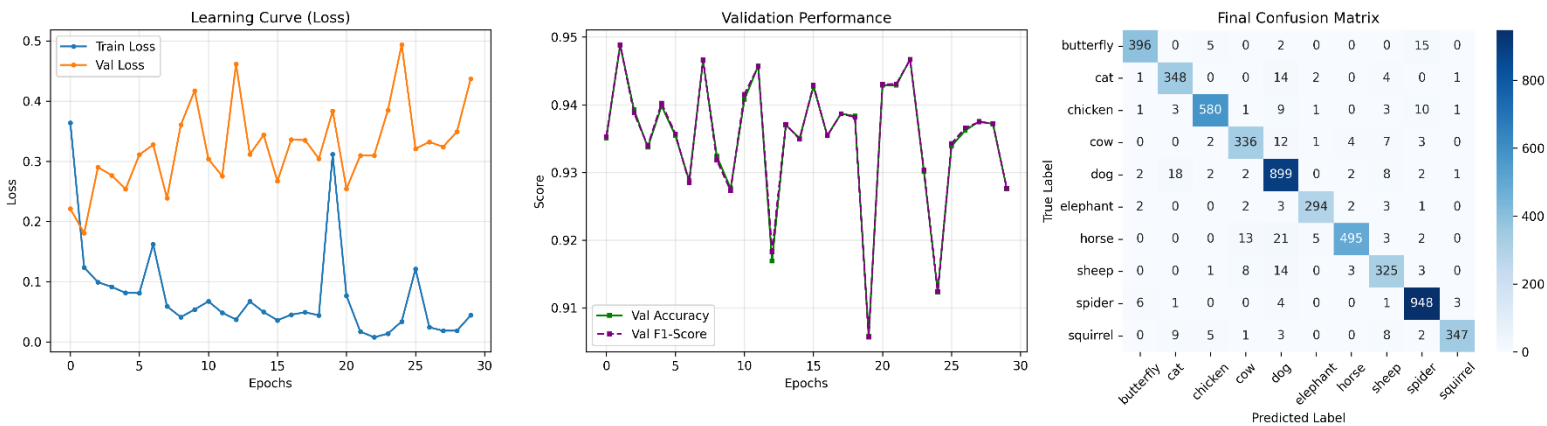
- Hiệu năng trên tập Val :

Mô hình đạt khoảng ~95.5% accuracy nhiều lần. Ngoại trừ đỉnh bất thường duy nhất, hiệu năng rất ổn định, dao động chặt chẽ trong khoảng 94.5%–95.5%.

- Ma trận nhầm lẫn :

Mô hình dự đoán đúng 933 ảnh “Dog”, cao hơn MobileNet (899) và tương đương ResNet (931). Điều này xác nhận rằng “compound scaling” của EfficientNet giúp nắm bắt các chi tiết tinh tế (như kết cấu lông, hình dạng tai) tốt hơn các CNN khác.

5. MobileNetV3



Ảnh 8. MobileNetV3

- Đường học :

Đường validation loss rất “nhiều” và gập khúc so với ResNet. MobileNet có số tham số ít hơn nhiều, nên khả năng “ghi nhớ” để làm mượt nhiều dữ liệu kém hơn. Mỗi batch khó đều tác động mạnh đến loss, khiến đường cong dao động lên xuống liên tục.

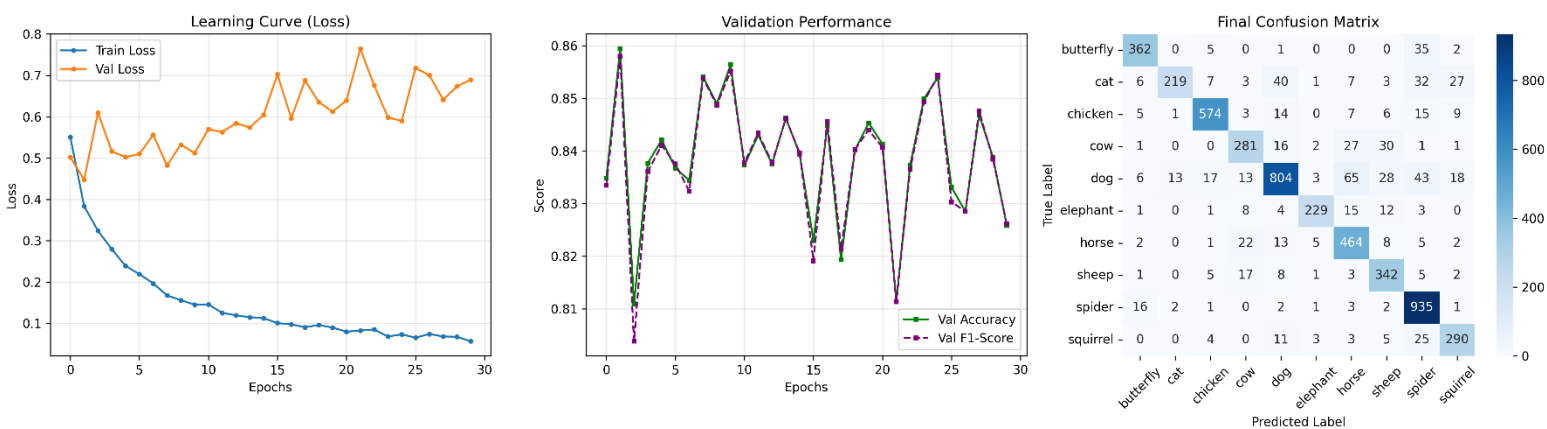
- Hiệu năng trên tập Val :

Accuracy dao động trong khoảng 91%–95%. Mặc dù accuracy trung bình cao, độ biến thiên lớn cho thấy trong triển khai thực tế, độ tin cậy có thể dao động nhẹ tùy thuộc vào ảnh đầu vào.

- Ma trận nhầm lẫn :

Mô hình nhầm 18 ảnh Cat thành Dog và 13 ảnh Horse thành Cow. Điều này cho thấy dù nắm bắt được hình dạng tổng quát, mô hình đôi khi bỏ sót các chi tiết kết cấu tinh tế giúp phân biệt các loài bốn chân có ngoại hình tương tự.

6. DeiT



Ảnh 9. DeiT

- Đường học :

Biểu đồ này minh họa rất rõ hiện tượng overfitting. Train Loss giảm xuống gần 0.05 trong khi Validation Loss tăng đều sau epoch 5, lên tới khoảng 0.7. Mô hình đã ghi nhớ các ảnh huấn luyện, nhưng không tổng quát hóa tốt sang ảnh mới và là vấn đề kinh điển của Transformer khi huấn luyện trên tập dữ liệu nhỏ (với ViT, 28.000 ảnh vẫn được xem là “nhỏ”).

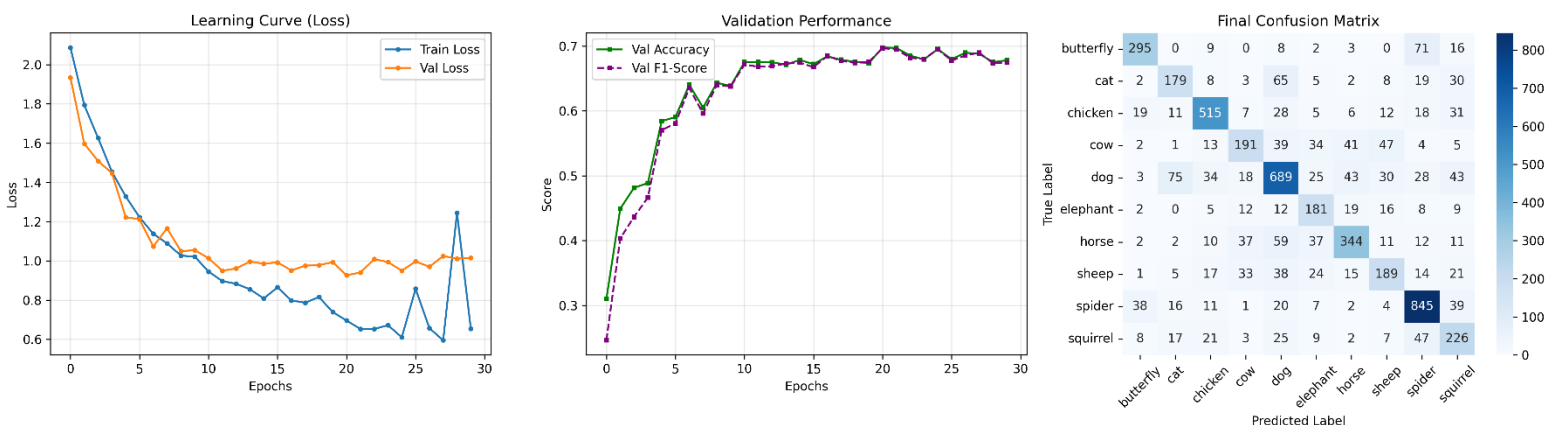
- Hiệu năng trên tập Val :

Accuracy bị kẹt quanh 84–86% và không cải thiện, dù training loss vẫn tiếp tục giảm.

- Ma trận nhầm lẫn :

Tỷ lệ nhầm lẫn cao giữa Cat và Dog (17 ảnh Cat bị phân loại thành Dog). Transformer thiếu “inductive bias” (không có hiểu biết sẵn rằng pixel tạo thành cạnh, hình dạng). Khi không có dữ liệu lớn, chúng gặp khó khăn trong việc phân biệt các lớp có ý nghĩa ngữ nghĩa gần nhau.

7. VGG16



Ảnh 10. VGG

- Đường học :

Đường validation loss gần như phẳng hoàn toàn sau 5 epoch đầu tiên. Do đóng băng backbone, mô hình không thể học các đặc trưng mới (ví dụ: “một con Spider trông như thế nào”). Nó chỉ có thể sử dụng các đặc trưng tổng quát đã học từ ImageNet.

- Hiệu năng trên tập Val :

Accuracy bị kẹt ở khoảng ~70%. Hiệu năng tương đương với Custom CNN, cho thấy rằng một mô hình lớn nhưng bị đóng băng và mang tính tổng quát chỉ xấp xỉ hiệu quả của một mô hình nhỏ được huấn luyện riêng cho bài toán cụ thể.

- Ma trận nhầm lẫn :

Đường chéo chính vẫn rõ ràng nhưng nhạt hơn nhiều (ít dự đoán đúng hơn) so với các mô hình tiền huấn luyện. Lỗi phân bố khá rộng. Có sự nhầm lẫn giữa các loài có shape giống nhau.

CHƯƠNG V. THIÊN HƯỚNG PHÁT TRIỂN

Hiện tại, dự án đang hướng tới việc phá bỏ rào cản phần cứng bằng cách thêm backend hỗ trợ cho các dòng GPU không sử dụng công nghệ CUDA (như AMD hoặc Apple Silicon), giúp mô hình chạy mượt mà trên nhiều nền tảng máy tính hơn. Bên cạnh đó, tính ứng dụng thực tiễn sẽ được đẩy mạnh thông qua việc thử nghiệm triển khai mô hình lên các thiết bị biên (edge devices) chuyên dụng để theo dõi động vật hoang dã hoặc tích hợp vào ứng dụng điện thoại di động. Điều này biến công cụ từ một mã nguồn nghiên cứu thành một sản phẩm "bỏ túi" hữu ích cho người dùng và các nhà bảo tồn.

Các mô hình tự thiết kế sẽ được phát triển để xử lý các bài toán phức tạp hơn, không chỉ dừng lại ở việc phân loại loài chung chung mà còn đi sâu vào nhận diện từng giống loài cụ thể (ví dụ: phân biệt các giống chó, mèo) và xác định chính xác vị trí của chúng trong khung hình. Đặc biệt, dự án sẽ tích hợp cơ chế phân loại theo bậc phân loại sinh học (loài, chi, họ, bộ...); cơ chế này giúp mô hình đưa ra dự đoán an toàn ở cấp độ "họ" hoặc "chi" nếu chưa chắc chắn về "loài", đảm bảo độ chính xác tổng thể cao hơn thay vì đoán sai hoàn toàn.

Tầm nhìn dài hạn của dự án là vượt ra khỏi khuôn khổ hình ảnh tĩnh để tiến tới xử lý dữ liệu đa phương thức, bao gồm nghiên cứu, phân tích video, âm thanh nhằm học được các đặc trưng về cử động và tiếng kêu của động vật. Để hiện thực hóa điều này, hệ thống sẽ cần được huấn luyện trên các tập dữ liệu quy mô lớn và chuyên sâu hơn như iNaturalist, Oxford-IIIT Pet, hay Caltech-UCSD Birds-200, giúp mô hình đạt được độ bao phủ rộng và khả năng tổng quát hóa tốt hơn trong môi trường tự nhiên.

KẾT LUẬN

Dự án này đã đánh giá thành công hiệu suất của bảy kiến trúc học sâu riêng biệt trên bộ dữ liệu Animals-10, làm rõ sự đánh đổi quan trọng giữa độ phức tạp của mô hình, độ chính xác và hiệu quả tính toán. Các kết quả đã xác định ResNet50 là kiến trúc vượt trội nhất, đạt độ chính xác cao nhất ~97.0% nhờ vào cơ chế residual learning. Trong khi các mạng nơ-ron tích chập (CNN) hiện đại như EfficientNetV2 và MobileNetV3 mang lại độ chính xác cạnh tranh với yêu cầu tài nguyên thấp hơn đáng kể, ta cũng thấy được những thách thức khi áp dụng Vision Transformers (DeiT, Custom ViT) cho các bộ dữ liệu quy mô trung bình, nơi chúng gặp khó khăn trong việc tổng quát hóa nếu không được huấn luyện trước trên dữ liệu lớn.

Hơn nữa, việc thực hiện các tối ưu hóa phần cứng—cụ thể là gradient checkpointing và đóng băng backbone—đã chứng minh vai trò thiết yếu trong việc huấn luyện các mô hình quy mô lớn như VGG16 trên phần cứng bị hạn chế (6GB VRAM). Cuối cùng, bằng cách tích hợp các mô hình này vào một quy trình hoàn chỉnh bao gồm cả giao diện GUI Python, ta hoàn toàn có thể triển khai các bài toán computer vision trên phần cứng phổ thông thông qua lựa chọn các kiến trúc kỹ lưỡng và kỹ thuật tối ưu hóa chiến lược.

TÀI LIỆU THAM KHẢO

- [1] <https://www.geeksforgeeks.org/machine-learning/metrics-for-machine-learning-model/>
- [2] <https://www.geeksforgeeks.org/machine-learning/macro-average-vs-weighted-average/>
- [3] <https://www.geeksforgeeks.org/machine-learning/introduction-convolution-neural-network/>
- [4] <https://www.geeksforgeeks.org/deep-learning/categorical-cross-entropy-in-multi-class-classification/>
- [5] <https://apxml.com/courses/advanced-pytorch/chapter-3-optimization-training-strategies/advanced-regularization>
- [6] <https://www.geeksforgeeks.org/deep-learning/why-is-adamw-often-superior-to-adam-with-l2-regularization-in-practice/>
- [7] <https://benihime91.github.io/blog/machinelearning/deeplearning/python3.x/tensorflow2.x/2020/10/08/adamW.html>
- [8] <https://cs231n.github.io/transfer-learning/>
- [9] <https://www.geeksforgeeks.org/computer-vision/efficientnet-architecture/>
- [10] <https://viso.ai/deep-learning/mobilenet-efficient-deep-learning-for-mobile-vision/>
- [11] <https://www.geeksforgeeks.org/machine-learning/depth-wise-separable-convolutional-neural-networks/>
- [12] <https://www.youtube.com/watch?v=w1UsKanMatM>
- [13] <https://arxiv.org/html/2411.13010v1>
- [14] <https://arxiv.org/abs/2010.11929>
- [15] https://huggingface.co/docs/transformers/model_doc/vit
- [16] <https://www.geeksforgeeks.org/deep-learning/vision-transformer-vit-architecture/>
- [17] <https://apxml.com/courses/how-to-build-a-large-language-model/chapter-11-scaling-transformers-architectural-choices/normalization-layer-placement>
- [18] <https://www.newline.co/@zaoyang/pre-norm-vs-post-norm-which-to-use--3ea6df8c>
- [19] <https://arxiv.org/pdf/2002.04745>
- [20] <https://medium.com/@fernandopalominocobo/demystifying-visual-transformers-with-pytorch-understanding-patch-embeddings-part-1-3-ba380f2aa37f>
- [21] https://d2l.ai/chapter_attention-mechanisms-and-transformers/vision-transformer.html
- [22] <https://apxml.com/courses/how-to-build-a-large-language-model/chapter-3-scaling-transformers-architectural-choices/choice-activation-functions>
- [23] https://vitalab.github.io/blog/2024/08/20/new_activation_functions.html

- [24] https://en.wikipedia.org/wiki/Residual_neural_network#Mathematics
- [25] <https://arxiv.org/abs/2510.24036>
- [26] <https://viblo.asia/p/paper-explain-hieu-ve-skip-connection-mot-ki-thuat-nho-ma-co-vo-trong-cac-kien-truc-residual-networks-3Q75w7bQ5Wb>
- [27] <https://lessw.medium.com/efficientnet-from-google-optimally-scaling-cnn-model-architectures-with-compound-scaling-e094d84d19d4>
- [28] <https://arxiv.org/pdf/1905.11946>
- [29] <https://viblo.asia/p/efficientnet-cach-tiep-can-moi-ve-model-scaling-cho-convolutional-neural-networks-Qbq5QQzm5D8>
- [30] <https://apxml.com/courses/cnns-for-computer-vision/chapter-1-cnn-foundations-modern-architectures/efficientnet-compound-scaling>
- [31] <https://www.emergentmind.com/topics/efficientnetv2-architecture>
- [32] <https://www.emergentmind.com/topics/efficientnetv2-architecture>
- [33] <https://research.google/blog/introducing-the-next-generation-of-on-device-vision-models-mobilenetv3-and-mobilenetedgepu/#:~:text=The%20Need%20for%20Hardware%2DAware%20Models&text=MobileNetV3%20is%20still%20the%20best,CPU%20as%20the%20deployment%20target.>
- [34] https://huggingface.co/docs/transformers/model_doc/deit
- [35] <https://arxiv.org/html/2404.02900v1#:~:text=In%20the%20Data%20Efficient%20Transformer,learns%20via%20distillation%20from%20the>
- [36] <https://www.geeksforgeeks.org/computer-vision/vgg-16-cnn-model/>
- [37] <https://arxiv.org/abs/1409.1556>
- [38] <https://www.kaggle.com/code/blurredmachine/vggnet-16-architecture-a-complete-guide>